

System Failure Report: Titanic and Machine Agents

A Comparative Analysis of Testing Limitations and Production Errors

By: Sulav Koirala (080BCT089)

Summary

This report looks at how big systems fail, comparing historical accidents with modern software problems. It starts by exploring why the Titanic was built, how it was tested, and why it failed in the real world. These events are compared to the software development life cycle to show how mistakes made early in design can stay hidden during testing but show up as critical issues during live use. By framing the sinking of the Titanic as an engineering and architectural failure, we can better understand how latent defects manifest in live, high-pressure environments.

The report also looks at automated systems in machines. It provides examples where automated software agents helped stop bad accidents from happening, and examples where a breakdown or wrong action by an automated agent caused severe damage and loss of life, such as the Boeing 737 MAX flight control software. Through these modern and historical studies, this paper establishes clear guidelines for designing, testing, and verifying complex autonomous systems to avoid catastrophic structural and algorithmic failures.

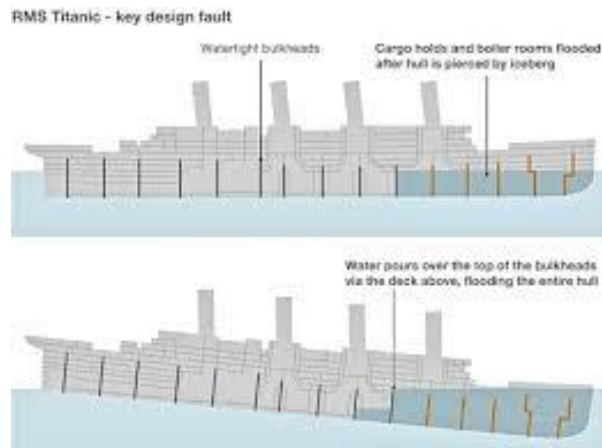
Titanic Background, Testing, and Use

Why the Ship Was Created

The Titanic was built because of intense business competition between two big cruise lines, White Star Line and Cunard Line, in the early 1900s. Cunard had built two very fast ships that dominated the Atlantic travel market. White Star Line knew it could not match that speed without expensive upgrades. Instead, they decided to focus on making much larger ships that offered total comfort,

luxury, and an image of absolute safety. This corporate positioning required an entirely new approach to ship construction, scaling dimensions beyond any prior maritime engineering framework.

The goal was to win over rich first-class travelers who wanted a luxury experience, while also carrying large numbers of third-class passengers who were moving to America. The ship used new safety features like a double-bottom hull and sixteen watertight compartments with doors that could close automatically if water was detected. This led experts at the time to claim the ship was practically unsinkable, creating a dangerous level of overconfidence throughout the White Star Line organization.



What Was Done During Testing

The testing phase for the Titanic was very short. The actual sea trials took place on April 2, 1912, and lasted less than eight hours. The tests were done in calm, safe waters near the shipyard. The crew tested basic steering, turned the ship in circles, measured speed, and did a crash stop where they threw the engines into reverse. These parameters only validated that the basic mechanical parts functioned under ideal conditions.

The testing did not include any difficult environments or emergency situations. The ship was never tested in freezing water, never dealt with simulated ice fields, and was never driven at full speed under heavy passenger weight. This simple testing gave the builders and crew a false sense of security, assuming that successful baseline performance guaranteed survival in extreme situations.

Real World Use

The real-world use began with the maiden voyage from England to New York. In the actual environment, commercial pressures pushed the captain to maintain high speeds to prove the ship's efficiency, even when receiving wireless warnings about ice fields ahead. Because the initial testing was successful and the design was praised as safe, the operators did not adjust their speed or route for

the changing environment. The live execution environment introduced critical threats that the initial test window completely ignored.

Why the Ship Failed in the Environment

The Titanic hit an iceberg on April 14, 1912, and sank in less than three hours. The failure happened because several deep engineering problems combined all at once in the real-world environment:

Material brittleness in cold water. The steel plates used for the hull contained high amounts of sulfur and phosphorus. In the freezing water of the North Atlantic, this chemical mix caused the metal to lose its flexibility. When it hit the iceberg, the steel cracked open instead of bending to absorb the impact. This material vulnerability was a latent defect completely unmapped during warm-water trials.

Weak iron rivets. The builders used lower-grade iron rivets in the front and back sections of the ship because automated riveting tools could not reach those areas. These manual iron rivets had low strength and easily snapped under the pressure of the collision, allowing the hull seams to rip open under dynamic water pressure.

Short watertight walls. The interior safety walls did not go all the way to the upper decks. They were left open at the top. As the first five compartments filled with water, the heavy bow tilted downward. This caused water to spill over the top of one wall into the next, like water filling an ice cube tray, rendering the watertight compartment defense completely useless.

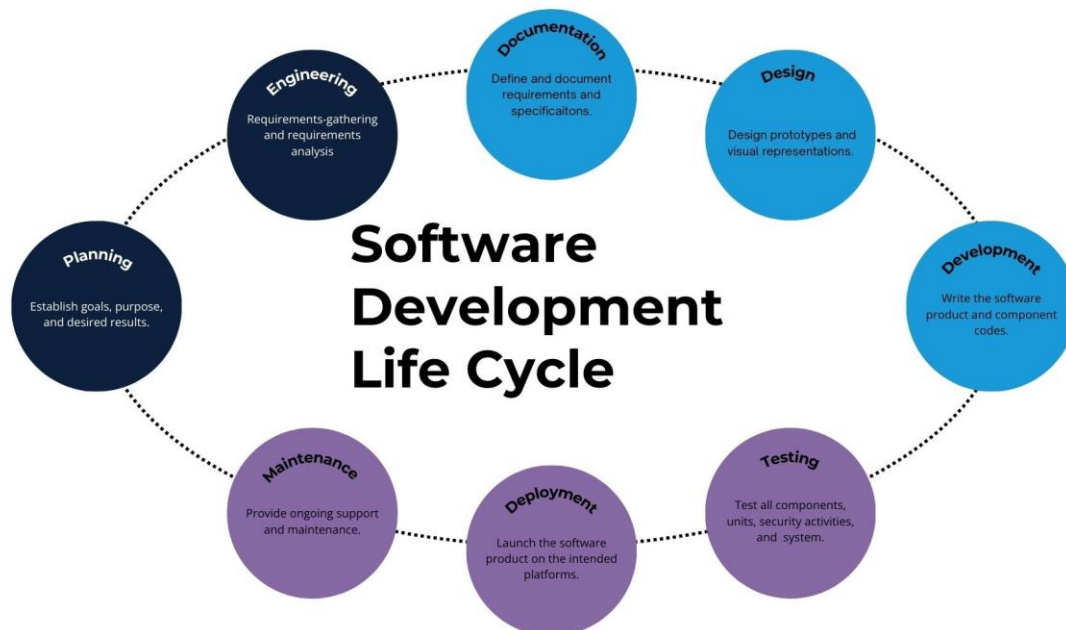
Not enough lifeboats. The ship carried only twenty lifeboats, which was legal under old safety laws but nowhere near enough for everyone on board. This choice was made to keep the walking decks looking clean and open, relying too much on the assumption that the ship itself acted as its own lifeboat, proving that marketing and style decisions compromised safety requirements.

Comparison to Software Development

The way the Titanic failed matches how modern software applications fail during the development life cycle. Architectural errors introduced during the early phases can easily get past tests and cause severe issues in production. When engineering teams rush deployment or underestimate operational stress, the exact same patterns of failure emerge.

Software Stage	Titanic Project Equivalence	Software Translation
Requirements	Choosing visual luxury and deck space over emergency lifeboat capacity.	Focusing heavily on user interface design and speed while skipping security and crash recovery rules.
Design	Designing short internal safety walls, assuming a major breach	Creating bad core architecture that assumes internal parts will never

	was impossible.	face an external attack or error.
Development	Buying lower-grade iron rivets and high-sulfur steel due to material shortages.	Using insecure third-party software code blocks just to finish the project quickly.
Testing	Running short trials in calm waters without testing extreme cold or heavy loads.	Testing only standard behaviors while ignoring stress tests, bad data inputs, and edge cases.
Production	Sailing into a freezing, ice-filled environment at full commercial speed.	Deploying code to live servers with heavy user traffic, active hackers, and high load spikes.



How Errors Hide During Testing

Severe defects often hide during testing because test environments are highly simplified versions of the real world. Staging environments and automated test scripts run under perfect conditions with clean data. They replicate the expected user behaviors while completely ignoring erratic, high-stress interaction patterns.

Errors slip into the live production environment due to several factors:

Environment mismatch. Just as the Titanic trials happened in smooth water, software test servers rarely show the messy nature of live use. Network delays, database bottlenecks, unexpected API

updates, and competing user requests are often missing during testing, allowing unstable builds to pass checks.

Hidden edge cases. Some errors need a highly specific set of circumstances to trigger. A code error might sleep until it hits a specific traffic spike combined with a particular user input, similar to how the Titanic steel only failed when it met freezing water. These environmental triggers are rarely replicated in standard test cycles.

Happy path testing bias. Teams often test if the software works when used correctly, rather than trying to break it with bad inputs or simulated failures. When tests only check basic functions, they miss deep architectural flaws. This functional compliance creates a deceptive metric of system readiness.

Automated Agents in Machinery

Modern systems use automated software agents to manage complex operations and fix problems faster than humans can. When built correctly, these agents protect systems by instantly executing corrections. However, if they suffer from design flaws or incorrect assumptions, their speed and autonomy can scale a localized anomaly into a catastrophic failure.

Examples of Agents Preventing Disasters

Automated systems have successfully stopped several dangerous industrial and transport accidents from happening, proving the value of closed-loop machine intelligence when properly bounded:

Nuclear power protection systems. In modern nuclear reactors, software agents monitor heat, pressure, and radiation. If these values go past safe levels, the automated system drops control rods into the reactor core within seconds to stop the reaction. This happens without waiting for human input, stopping meltdowns during sudden grid or mechanical shifts.

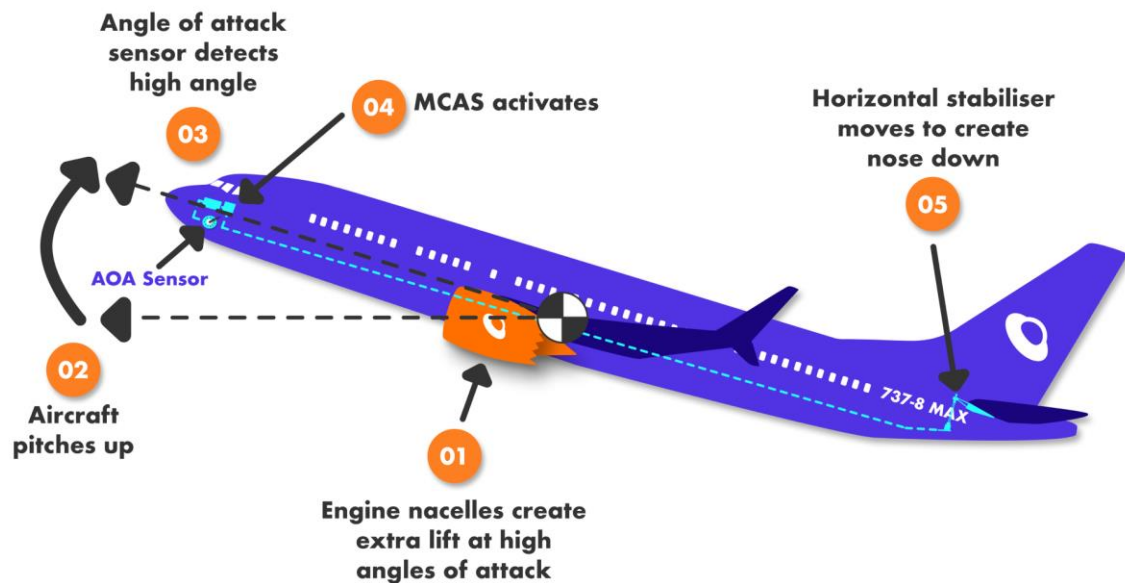
Aircraft collision avoidance systems. Aircraft use an onboard agent that talks to nearby planes. If two planes are on a collision course, the agent calculates a fix and gives direct steering orders to the pilots, telling one to climb and the other to descend, bypassing slower human air traffic control to resolve immediate airspace conflicts.

Power grid isolation systems. When a storm or short circuit damages power lines, automated agents detect the electrical spike in milliseconds. The system cuts off the damaged section immediately, keeping the rest of the grid stable and stopping a massive multi-state blackout through automated islanding protocols.

Examples of Agent Failures Causing Loss

When automated agents have design flaws or get wrong sensor data, their speed and power can lead to severe accidents, bypassing human operator intent completely:

The Boeing 737 MAX flight control software. The crashes of Lion Air Flight 610 and Ethiopian Airlines Flight 302 happened due to a flaw in the MCAS automated flight agent. Built to stop the plane from stalling, the software relied on just one sensor. When that sensor failed, the agent wrongly thought the plane was stalling and repeatedly pushed the nose down. The pilots could not stop the automated system, resulting in the loss of 346 lives.



The Knight Capital stock trading crash. An automated financial trading agent was active on live servers with old code left inside it. When the market opened, a minor setup mistake caused the agent to get stuck in a loop, buying and selling millions of stocks in minutes. The system lost 440 million dollars in 45 minutes and ruined the company, showing how fast unconstrained software loops can destroy equity.

The autonomous vehicle pedestrian accident. A self-driving test car struck a pedestrian in Arizona. The vehicle's software agent failed to classify a person walking a bicycle across the street. The agent kept changing its guess between a vehicle, a bike, and a person, resetting its memory each time. This logic error delayed the automated brakes until it was too late, proving that classification instability can kill.

References

- Boeing Flight Control Software Investigation, Federal Aviation Administration Report, 2019.
- Garber, Nicholas. Metallurgical Analysis of Hull Steel Plate Recovered from the RMS Titanic Wreckage. Materials Engineering Press, 1998.
- Knight Capital Group Systems Trading Error Review, Securities and Exchange Commission Release, 2012.
- National Transportation Safety Board. Collision Between a Self-Driving Car and a Pedestrian in Tempe, Arizona. Accident Report NTSB/HAR-19/03, 2019.
- Sommerville, Ian. Software Engineering and System Architecture Lifecycle Management. Pearson Education, 2015.
- White Star Line Construction Records and Sea Trial Logs, Harland and Wolff Archives, Belfast, 1912.